Politecnico di Torino

Department of Control and Computer Engineering

Master's Degree in Computer Engineering

2025/2026

Technologies for Autonomous Vehicles

Assignment 1: Global Path Planning

17/03/2026

Fabio Veroli – S336301

Contents

1 Objective **2**

2 Design **2**
2.1 Graph Construction 2
2.2 Edge Weight Computation 2
2.3 Dijkstra's Algorithm 2
2.4 A* Algorithm 3

3 Experiments **4**
3.1 Datasets 4
3.2 Experimental Protocol 4
3.3 Evaluation Metrics 4

4 Results and Discussion **5**
4.1 Aosta 5
4.2 Turin 6
4.3 Discussion 7

1 Objective

This report presents the implementation and comparative analysis of two global path planning algorithms, Dijkstra's algorithm and A*, applied to real-world road networks. The primary objective is to evaluate and compare the computational performance of the two algorithms in terms of the number of iterations required to find an optimal path, across two cities of significantly different scale: Aosta and Turin.

Dijkstra's algorithm is used as the baseline, as it guarantees an optimal solution on graphs with non-negative edge weights. A* is then implemented in three variants, each employing a different heuristic function to guide the search:

- **Manhattan distance**
- **Euclidean distance**
- **Haversine distance**

For each algorithm and city, ten origin-destination pairs are evaluated and the average number of iterations is recorded. The results are then discussed in relation to graph size and heuristic choice, with the aim of assessing the practical benefit of informed search over uninformed search.

2 Design

2.1 Graph Construction

Road networks are obtained from OpenStreetMap using the `osmnx` library. For each city, a directed multigraph is constructed, each node represents a road junction and stores geographic coordinates (longitude and latitude), while each directed edge represents a road segment with attributes such as `length` (in metres) and `maxspeed` (in km/h).

2.2 Edge Weight Computation

The weight assigned to each edge models travel time rather than physical distance, as travel time is a more practical optimization criterion for vehicle routing. The weight assigned to each edge is:

$$w(e) = \frac{\ell(e)}{v_{\max}(e)}$$

where $\ell(e)$ is the length of edge e in metres and $v_{\max}(e)$ is the posted speed limit converted to m/s. The resulting weight equals the true travel time $t(e)$ in seconds; the algorithm therefore minimizes total travel time directly. For edges where no speed limit is recorded in OpenStreetMap, a default value of 40 km/h is assumed.

2.3 Dijkstra's Algorithm

Dijkstra's algorithm is implemented using a binary min-heap as the priority queue. Each node maintains a `distance` label, initialized to $+\infty$ except for the source, which is set to 0. At each iteration, the node with the smallest tentative distance is extracted; if it has already been marked as visited, it is skipped; otherwise, all outgoing neighbors are

relaxed. The algorithm terminates as soon as the destination node is extracted from the queue, guaranteeing optimality on graphs with non-negative edge weights.

2.4 A* Algorithm

A* extends Dijkstra by incorporating a heuristic function $h(n)$ that estimates the remaining cost from node n to the destination. The priority of each node in the queue is:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the exact travel-time cost accumulated from the source to n. Three functions are used to compute the geometric distance d:

- **Manhattan**: $d = |x_1 - x_2| + |y_1 - y_2|$
- **Euclidean**:
$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$
The computation is delegated to `ox.distance.euclidean`.
- **Haversine**: the great-circle distance between two points on the Earth's surface, given their latitude ϕ and longitude λ (the formula is expressed in radians; the implementation via `ox.distance.great_circle` accepts decimal degrees and performs the conversion internally):
$$\begin{aligned} a &= \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \, \cos\phi_2 \, \sin^2\left(\frac{\Delta\lambda}{2}\right) \\ c &= 2\,\mathrm{atan2}\left(\sqrt{a},\, \sqrt{1-a}\right) \\ d &= R\,c, \quad R = 6371 \text{ km} \end{aligned}$$
The computation is delegated to `ox.distance.great_circle`, which operates on geographic (lat/lon) coordinates.

To ensure dimensional consistency between $g(n)$ and $h(n)$, each distance-based heuristic estimate is normalized by the maximum speed limit $v^\star_{\max}$ observed in the graph:

$$h(n) = \frac{d(n,\, \text{dest})}{v^\star_{\max}}$$

For the Euclidean and Haversine variants, this normalization guarantees admissibility: since any road path between two nodes is at least as long as the straight-line distance between them, dividing the straight-line estimate by $v^\star_{\max}$ yields a lower bound on the remaining travel time. The Manhattan variant satisfies $d_{\text{Man}} \geq d_{\text{Euc}}$ and therefore does not carry a formal admissibility guarantee.

All three variants share the same min-heap structure, early-termination condition, and visited-node logic as the Dijkstra implementation.

To ensure that the heuristics $h(n)$ are on the same scale as the accumulated edge weights $g(n)$ (in seconds), the geometric distances must be computed in metres before being normalized by $v^\star_{\max}$. For the Haversine variant, this is handled natively: `ox.distance.great_circle` accepts geographic coordinates (latitude and longitude in

decimal degrees) and always returns a distance in metres. For the Manhattan and Euclidean variants, however, computing distances directly from latitude and longitude differences would yield values in degrees (10^{-3}–10^{-2}), which become negligible after normalization, effectively causing A* to degenerate to Dijkstra's algorithm.

To resolve this, the Manhattan and Euclidean variants are evaluated on a projected version of the graph. A projected graph is obtained by converting the original geographic coordinates to a planar metric coordinate reference system (CRS). Concretely, `ox.project_graph` projects all node coordinates to the appropriate local UTM zone, replacing degree-based values with positions in metres; as a side effect, edge lengths are recalculated in the new coordinate system. On the projected graph, the x and y coordinate differences used by the Manhattan and Euclidean functions are correctly expressed in metres. The effect of this coordinate projection on algorithm performance is evaluated in Section 4.1.

3 Experiments

3.1 Datasets

Experiments are conducted on the road networks of two cities: Aosta and Turin. These cities are selected to represent graphs of very different scales, allowing the comparison of algorithm behavior under both small and large network conditions. Road networks are downloaded and loaded using `osmnx` version 2.1.0. The graph sizes are reported in Table 1.

City	Nodes	Edges
Aosta	825	1 723
Turin	11 766	25 198

Table 1: Size of the road network graphs used in the experiments.

3.2 Experimental Protocol

For each city, ten origin-destination pairs are randomly sampled from the set of graph nodes. Dijkstra's algorithm is executed first; any pair for which no path exists is discarded and a new pair is sampled, so that all ten runs correspond to valid paths. The ten validated pairs are then saved and reused for all three A* variants, ensuring that every algorithm is evaluated on identical inputs and that the results are directly comparable.

3.3 Evaluation Metrics

Three metrics are recorded for each run:

- **Number of iterations**: the count of nodes successfully extracted from the priority queue, marked as visited, and whose outgoing neighbors are relaxed, before the destination node is extracted. This is the primary metric for comparing the computational efficiency of the algorithms.

- **Path length** (km): the total length of the reconstructed path in kilometers. This metric is used to verify that the paths returned by A* are optimal, i.e., identical to those returned by Dijkstra on the same pair.
- **Travel time** (s): the total travel time of the reconstructed path in seconds, computed as the sum of the edge weights along the path. Since the edge weight represents the physical travel time, this metric provides a second, independent check of solution optimality: an A* path is confirmed optimal when both its length and its travel time match those of the Dijkstra solution.

The average number of iterations over the ten runs is used as the summary statistic for each algorithm–city combination.

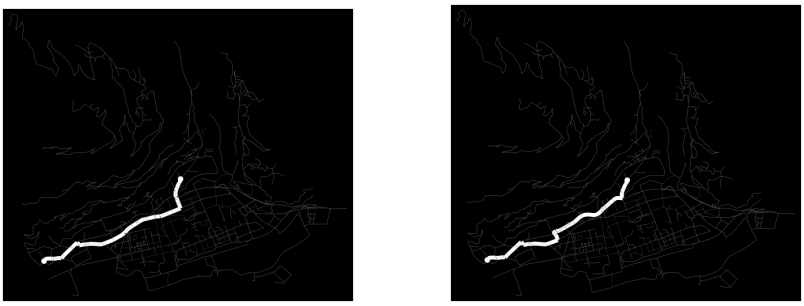
4 Results and Discussion

4.1 Aosta

Table 2 reports the number of iterations per run for the four algorithm configurations on the Aosta network. The A* variants all reduce the number of iterations meaningfully compared to Dijkstra: Manhattan achieves the largest reduction, while Euclidean and Haversine perform similarly. Paths and travel times are identical across all algorithms and runs, with one exception: on run 9, A* Manhattan found a suboptimal path (2.8683 km, 226.40 s) compared to the optimal solution found by all other variants (2.8811 km, 222.97 s), marked with † in the table. Figure 1 shows the paths produced on run 9, chosen as the representative example since it exhibits this divergence.

Run	**Dijkstra**	**Manhattan**	**Euclidean**	**Haversine**	**Dist. (km)**	**Time (s)**
1	189	69	87	87	1.1034	112.11
2	425	170	244	244	1.6301	182.85
3	615	220	269	269	2.6132	240.84
4	252	27	44	44	1.0240	104.38
5	748	433	569	569	5.0599	410.44
6	761	577	628	628	4.0024	355.85
7	106	13	13	14	0.9574	68.93
8	288	76	108	108	1.5966	141.76
9	633	55†	133	133	2.8811	222.97
10	23	14	16	16	0.6414	54.49
Avg	404.0	165.4	211.1	211.2		
Impr.	—	−59.1%	−47.7%	−47.7%		

Table 2: Iterations per run and average improvement over Dijkstra - Aosta (825 nodes, 1 723 edges). Dist. and Time refer to the optimal path found by Dijkstra and all admissible variants; † marks a suboptimal result.



(a) Dijkstra

(b) A* Manhattan

(c) A* Euclidean

(d) A* Haversine

Figure 1: Paths returned by each algorithm on run 9 in Aosta. The Manhattan variant found a suboptimal path (†); all other variants return the optimal solution.

4.2 Turin

Table 3 reports the results for the Turin network, which is roughly 14 times larger than Aosta. Manhattan, Euclidean, and Haversine all achieve meaningful reductions compared to Dijkstra. Path lengths and travel times are identical across all algorithms for every run, confirming that all variants find the same optimal path.

Run	**Dijkstra**	**Manhattan**	**Euclidean**	**Haversine**	**Dist. (km)**	**Time (s)**
1	9 927	6 314	7 128	7 135	8.8877	633.13
2	5 100	2 775	3 230	3 232	5.6477	423.60
3	7 508	5 773	6 379	6 380	9.2594	697.13
4	2 233	813	1 004	1 004	3.6459	268.79
5	8 985	4 762	5 373	5 375	6.9509	528.00
6	6 249	3 926	4 479	4 481	7.0667	525.64
7	4 679	1 888	2 383	2 383	3.7012	329.82
8	4 399	2 939	3 089	3 093	8.3023	625.26
9	5 947	2 815	3 197	3 197	7.0070	534.06
10	4 001	1 150	1 653	1 655	4.3905	316.11
Avg	5 902.8	3 315.5	3 791.5	3 793.5		
Impr.	—	−43.8%	−35.8%	−35.7%		

Table 3: Iterations per run and average improvement over Dijkstra — Turin (11 766 nodes, 25 198 edges).

4.3 Discussion

Path lengths and travel times are identical across all algorithm-run combinations in both cities, confirming optimal solutions on every run. The sole exception is A* Manhattan on run 9 in Aosta, which found a suboptimal path. This is a direct consequence of the Manhattan heuristic not being admissible: by overestimating the remaining cost on run 9, A* discards the true optimal branch before visiting it, committing to a suboptimal solution. A* Euclidean, which uses the same projected graph but with an admissible and consistent heuristic, correctly recovers the optimal solution.

Manhattan achieves the largest reduction in expanded nodes: -59.1% in Aosta and -43.8% in Turin. Euclidean and Haversine perform similarly in both cities: in Aosta, -47.7% and -47.7% respectively; in Turin, -35.8% and -35.7%. The near-identical performance of Euclidean and Haversine is expected: at the spatial scale of a single city, the Earth's curvature is negligible, and the great-circle distance computed by the Haversine formula closely approximates the Euclidean distance in a planar setting.

The larger reduction achieved by Manhattan compared to Euclidean is consistent with the known behavior of overestimating heuristics: by assigning higher $f(n)$ values to nodes farther from the goal, Manhattan focuses the search more aggressively, at the cost of admissibility, as evidenced by the suboptimal result on run 9 in Aosta.

The experiments demonstrate that A* can substantially reduce the number of node expansions required to find a shortest path in road networks, regardless of graph scale. The choice of heuristic plays a crucial role: Manhattan provides the most aggressive pruning, but at the cost of the optimality guarantee; Euclidean and Haversine offer a good trade-off, achieving meaningful reductions while preserving correctness.